FAB : Framework for Analyzing Benchmarks

Shreyas Singh*
shreyas.singh@iitgn.ac.in
Indian Institute of Gandhinagar

Varun Gohil*
gohil.varun@iitgn.ac.in
Indian Institute of Gandhinagar

Manu Awasthi
manu.awasthi@ashoka.edu.in
Ashoka University

ABSTRACT

Computer architecture research hinges on the ability of architects to select the right set of workloads required to demonstrate the benefits of proposed enhancements to existing architectures. During the initial design phases, architects usually test their proposals on a subset of representative workloads to gain early feedback and insights. However, due to the large number of benchmark suites present today, combined with the complexity and time cost associated with running those benchmarks, choosing and simulating workloads for design trade-off evaluations has become extremely difficult.

In this paper, we present FAB - a framework built with Pin and Python-based workflow. FAB allows a user-driven analysis of benchmarks across multiple axes like instruction distributions and types of instructions through an interactive Python interface. FAB aims to provide a toolkit that 1) acts as a central repository of workload characteristics over multiple benchmarks, and 2) uses these characteristics to perform a user-driven analysis of workload across multiple benchmark suites.

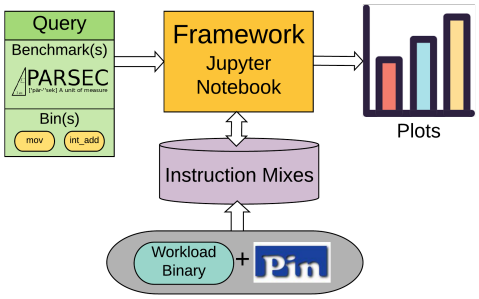
1 INTRODUCTION

Performance evaluation is at the foundation of computer architecture research and development. Researchers must perform rigorous analysis of their architectures to steer their research in the right direction. Researchers typically use benchmark suites for quantifying the performance of a system or a subsystem. These benchmarks suites consist of several workloads which are representative of a particular application domain. However, in the initial design stages, architects prefer measuring the benefits gained by testing the design only on a few workloads which are representative of the entire benchmark suite. This approach significantly reduces the initial testing time, which is essential for receiving early feedback. [4]

Today there exist a myriad of benchmark suites with each suite consisting of multiple workloads targeting various application domains. Moreover, even a single workload may have multiple input sets and behaviours. Because of this, choosing a representative set of workloads for a specific research project is not trivial.

*Both authors contributed equally to this research.

Figure 1: Current Framework



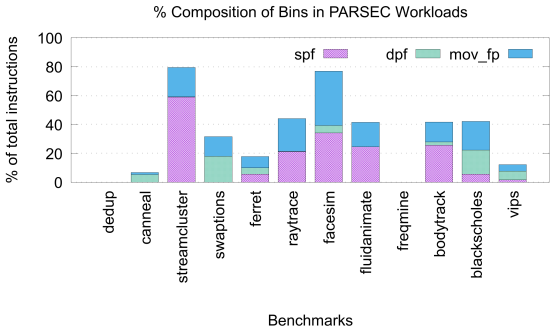
For this purpose, we introduce FAB which aims to facilitate the process of workload selection and analysis by acting as a central repository which can help architects compare workload characteristics across not just different workloads but across multiple suites in one place. The significant contributions of our framework are as follows:

- FAB provides a mechanism of instruction classification for the x86_64 instruction set. We manually classify the entire instruction set in to multiple bins, with each bin representing a set of instructions with similar functionality. These bins acts as axes for analysis and comparison of workloads. We also allow users to create their own custom bins.
- FAB also provides an interactive Python notebook to visualize the architectural characteristics of multiple workloads, from across benchmark suites. Visualizations like dendrograms, barcharts make analysis and comparison of workloads relatively simpler.

2 DESCRIPTION OF FAB

As shown in Figure 1, FAB's workflow currently consists of two parts. The first part, or the back-end, deals with an Intel Pin [6] based flow, which takes a workload binary and the associated input files as input, and generates an instruction mix for the workload as output. This has to be done once for every new workload. The instruction mix is then used for various kinds of analyses using the tool's front-end, which is a Jupyter notebook. The front-end also contains the pre-classified instruction bins, which can be referenced directly from within the notebook. The notebook takes the benchmark and instruction bins as input and produces stacked

Figure 2: Floating Point operations in PARSEC



barcharts and dendrograms which assist in analysis. Currently, FAB contains instruction mixes of **SPEC CPU 2017** [2], **PARSEC 3.0** [3], **SPLASH 3** [5] and **OpenBLAS** [1] benchmark suites.

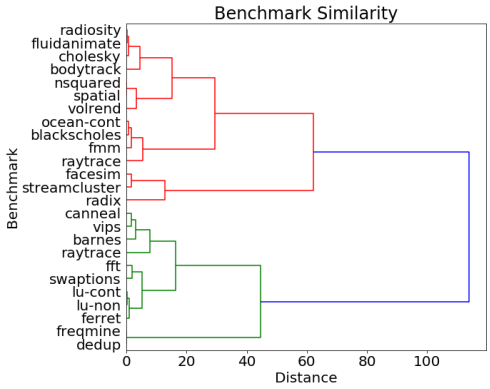
Instruction Mixes: FAB uses the instruction mixes as the primary characteristic for facilitating analysis. Instruction mixes are a microarchitecture independent characteristic that include the various instructions executed by a workload and their frequency. It is because of this feature, it gives us a very interesting way to analyse the different behaviours of benchmarks without having to worry about the underlying architecture. Moreover, as Intel Pin (using the *opcodemix* tool) allows us to instrument all the instructions executed by a program, we do not have to restrict oursleves to high level metrics like memory and branch operations [7].

Instruction Binning: To further facilitate analysis we allow users to group related instructions together into *bins* based on their own needs. These bins serve as well defined axes which can help architects analyze workloads along specified axis and choose the workloads as per one's requirements. As an example, an architect studying the energy differences in moving FLOAT and INT values would choose separate bins while using FAB. We also provide users with **43** premade bins that were manually classified by going through the x86 instruction set.

3 USING FAB

Workload Selection: Consider the example of architects who work on hardware for approximate computing. For such architects it is imperative to choose workloads that are amenable to the approximations introduced by their models - an example of which would be choose workloads that show high amount of floating point activity (like floating point arithmetic and moves. Using FAB to evaluate the different workloads in PARSEC we obtain a chart much like Fig. 2. This figure shows the distribution of Single-Precision, Double-Precision and Floating-Point Move operations happening in each workload. Using this the architects can choose those workloads that satisfy their criteria for workloads.

Figure 3: Dendrogram of Floating Point operations in PARSEC and SPLASH



Workload Similarity: For architects trying to reduce the number of experiments workload similarity plays a very important role in choosing workloads. For finding similarity between workloads, the framework uses agglomerative hierarchical clustering based on the percentage frequency of bins as specified by the user. Using this mechanism one can get workload sets with each workload having similar behavior for the specified bins. This allows users to select workloads from different sets so as to select workloads with diverse behaviors and increase coverage. We show the similarity of benchmarks for the floating point operation frequencies of PARSEC and SPLASH in Fig. 3.

4 ABOUT THE CONFERENCE

The International Conference on Performance Engineering (ICPE) is the leading international forum for presenting and discussing novel ideas, innovations, trends and experiences in the field of performance engineering. The conference aims to bring together researchers and practitioners to report state-of-the-art and in-progress research on performance engineering of software and systems while organizing tutorials and workshops for the same.

REFERENCES

[1] [n.d.]. OpenBLAS : An optimized BLAS library. http://www.openblas.net/
[2] [n.d.]. SPEC CPU 2017 Documentation. https://www.spec.org/cpu2017/
[3] Christian Bienia et al. 2008. *The PARSEC Benchmark Suite: Characterization and Architectural Implications.* Technical Report TR-811-08. Princeton University.
[4] Lieven Eeckhout. 2010. *Computer Architecture Performance Evaluation Methods* (1st ed.). Morgan & Claypool Publishers.
[5] C. Sakalis et al. 2016. Splash-3: A properly synchronized benchmark suite for contemporary research.
[6] Chi-Keung Luk et al. 2005. Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation. In *Proceedings PLDI.*
[7] R. Panda et al. 2018. Wait of a Decade: Did SPEC CPU 2017 Broaden the Performance Horizon?. In *Proceedings of HPCA.*